

MANUAL DE IMPLEMENTACIÓN

Experiencia de Presale – Gaia Ecotrack

Confirmación de compra, vesting, explorador de Solana y reclamo de tokens

PARA: Equipo de Desarrollo (Nico y equipo)

DE: Oficina de Asesoría Jurídica / Producto – Gaia Ecotrack

FECHA: 13 de agosto de 2026

VERSIÓN: 3.0 – Incluye experiencia de Claim

”La confianza se construye en el momento de la compra y se consolida en el momento del reclamo.”

Confidencial – Uso interno de Gaia Ecotrack

Índice

1. Objetivo y Contexto	4
1.1. Propósito del Documento	4
1.2. Contexto del Proyecto	4
1.3. Principios Rectores de la Experiencia	5
2. Flujo de Compra y Confirmación	6
2.1. Diagrama de Flujo de Alto Nivel	6
2.2. Captura del TX ID	7
2.3. Backend: Almacenamiento de la Transacción	7
3. Mensajes de Confirmación	9
3.1. Mensaje de Éxito	9
3.1.1. Contenido Obligatorio	9
3.1.2. Ejemplo Visual del Mensaje de Éxito	10
3.2. Mensaje de Error	10
3.2.1. Ejemplo Visual del Mensaje de Error	10
3.3. Estados de Error Comunes y Mensajes Asociados	11
3.4. Mensaje de Estado "Pendiente" (Timeout)	11
4. Enlace al Explorador y Datos On-Chain	13
4.1. Elección del Explorador	13
4.2. Generación del Enlace	13
4.3. Visualización del Enlace en la UI	13
4.4. Datos Adicionales On-Chain	14
5. Gestión del Vesting (Streamflow)	15

5.1. Integración con Streamflow	15
5.2. Flujo de Creación del Vesting	15
5.3. Enlace al Contrato de Streamflow	15
5.4. Mensaje Sugerido para el Vesting	16
6. Dashboard del Usuario ("Mi Inversión")	17
6.1. Propósito	17
6.2. Contenido del Dashboard	17
6.3. Requisitos Técnicos	17
7. Consideraciones Técnicas para Meteora	19
7.1. Integración con Meteora	19
7.2. Manejo de Slippage	19
7.3. Webhooks y Callbacks	19
7.4. Manejo de Fallos del RPC y Webhooks	20
7.4.1. Flujo de Confirmación con Polling	20
7.4.2. Implementación Técnica	20
7.5. Seguridad	21
8. Plan de Pruebas y Lanzamiento	22
8.1. Fases de Pruebas	22
8.2. Checklist de Lanzamiento	22
8.3. Roles y Responsabilidades	23
9. Experiencia de Reclamo (Claim) en el Dashboard	24
9.1. Objetivo del Capítulo	24
9.2. Indicadores Visuales de Estado y Tiempo	24
9.2.1. Contador Regresivo Personalizado (Por Wallet)	24

9.2.2. Barra de Progreso de Vesting	24
9.3. Interfaz de Acción (Smart Buttons)	25
9.3.1. Botón de "Claim" Dinámico	25
9.3.2. Botón "Añadir Token a la Wallet" (One-Click)	26
9.4. Transparencia y Prevención de Errores	26
9.4.1. Historial de Reclamos	26
9.4.2. Alerta de Saldo para Gas	27
9.4.3. Actualización Automática del Estado	27
9.5. Ejemplo Visual del Dashboard de Claim	27
9.6. Requisitos Técnicos para Desarrollo (Nico y equipo)	28
9.6.1. Lectura de Streamflow	28
9.6.2. Manejo de Errores en Claim	28
9.6.3. Actualización en Tiempo Real	29
9.7. Consideraciones de Seguridad	29
10. Anexos	30
10.1. Anexo A: Ejemplo de Mensaje de Confirmación de Compra (Copy Exacto)	30
10.2. Anexo B: Estructura de Datos para el Backend	30
10.3. Anexo C: Estructura de Datos para el Historial de Reclamos	31
10.4. Anexo D: Enlaces Útiles para Implementación	31
11. Referencias y Documentos Relacionados	32

1 Objetivo y Contexto

1.1 Propósito del Documento

El presente manual describe la experiencia de compra en la preventa pública de GAIA, con especial énfasis en:

- La confirmación visual de la compra exitosa.
- La visualización del estado del **vesting** (bloqueo y liberación escalonada).
- El enlace directo al **explorador de Solana** (Solscan) para verificar la transacción on-chain.
- El **dashboard personal** del usuario donde podrá consultar su inversión y el estado del vesting en cualquier momento.
- La **experiencia de reclamo (claim)** de tokens liberados, con indicadores visuales, botones inteligentes y transparencia total.

Este documento es la guía de implementación para Nico y el equipo de desarrollo. No es un documento conceptual; es una especificación técnica y de producto para ejecución.

1.2 Contexto del Proyecto

Gaia Ecotrack es una plataforma de tokenización de energía renovable sobre Solana. La preventa pública de GAIA se realizará a través de **Meteora** (DEX con pools dinámicos). El token GAIA está sujeto a un esquema de vesting:

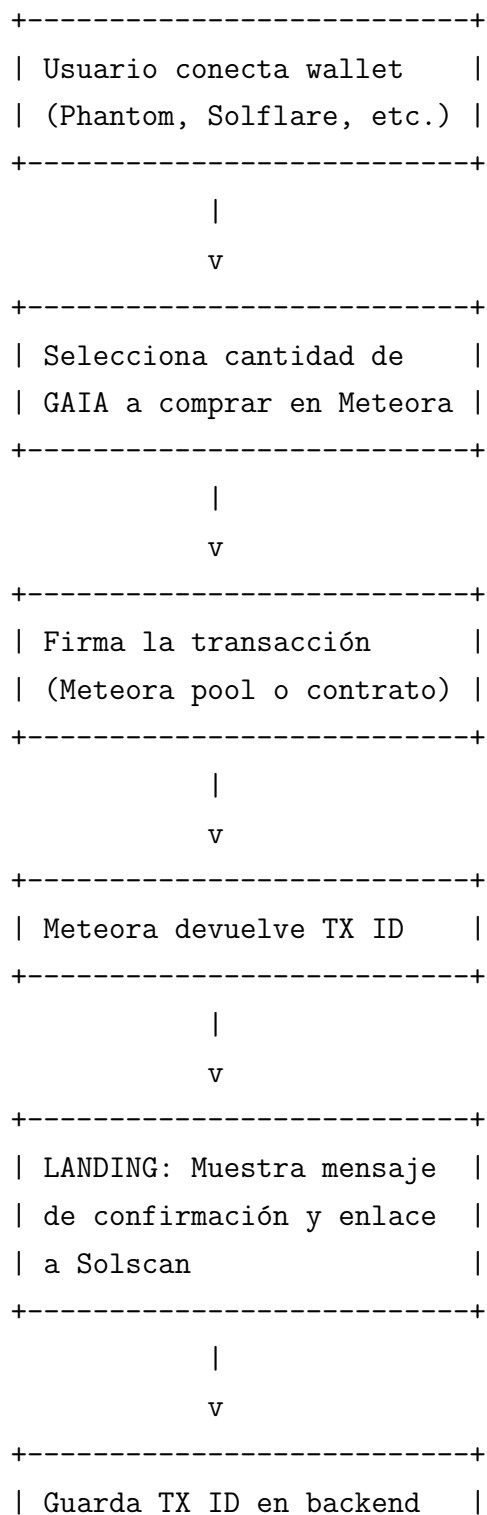
- **Lock de 6 meses:** Los tokens no se pueden mover ni vender durante los primeros 6 meses posteriores al Token Generation Event (TGE).
- **Liberación escalonada:** Después del lock, se libera el 25 % del total cada mes durante **4 meses** (iniciando en el mes 6 post-TGE: meses 6, 7, 8 y 9).
- **Gestión del vesting:** Se utilizará **Streamflow** para gestionar los contratos de bloqueo de tokens.

1.3 Principios Rectores de la Experiencia

1. **Transparencia total:** El usuario debe poder verificar cada paso de su compra y reclamo en la blockchain.
2. **Reducción de ansiedad:** Inmediatamente después de firmar la transacción, debe recibir una confirmación clara y visualmente destacada. Durante el vesting, debe saber exactamente cuándo podrá reclamar.
3. **Autonomía:** El usuario debe poder consultar el estado de su inversión y realizar reclamos en cualquier momento, sin depender del equipo de soporte.
4. **Profesionalismo:** La experiencia debe diferenciar a Gaia Ecotrack de proyectos que solo muestran un ”¡Gracias!” genérico.

2 Flujo de Compra y Confirmación

2.1 Diagrama de Flujo de Alto Nivel



```

| y vincula al usuario      |
+-----+
|
| v
+-----+
| Dashboard del usuario    |
| muestra detalles y       |
| estado del vesting       |
+-----+
|
| v
+-----+
| Cuando se desbloquean    |
| tokens, usuario puede    |
| reclamar (Claim) desde   |
| el dashboard             |
+-----+

```

2.2 Captura del TX ID

La transacción se ejecuta en Meteora. Después de la firma, Meteora devuelve un objeto de respuesta que contiene el **TX ID** (transacción hash). La aplicación debe:

1. Capturar este TX ID.
2. Enviarlo al backend para asociarlo al usuario (con su wallet pública).
3. Mostrarlo inmediatamente en la interfaz de confirmación.
4. Utilizarlo para generar el enlace a Solscan.

2.3 Backend: Almacenamiento de la Transacción

El backend debe almacenar la siguiente información para cada transacción de compra:

Cuadro 1: Estructura de Datos de la Transacción

Campo	Descripción
wallet	Dirección pública de la wallet del usuario
tx_id	Hash de la transacción en Solana
amount_gaia	Cantidad de GAIA adquiridos (en unidades enteras, sin decimales)
amount_usdc	Monto pagado en USDC
timestamp	Fecha y hora de la transacción (UTC)
vesting_schedule	Objeto con fechas de lock y liberaciones

3 Mensajes de Confirmación

3.1 Mensaje de Éxito

3.1.1 Contenido Obligatorio

Inmediatamente después de la transacción exitosa, la landing debe mostrar un mensaje con los siguientes elementos:

1. **Encabezado visual:** Un ícono de éxito (checkmark verde grande) y el texto "¡Compra exitosa!" en tamaño destacado.
2. **Resumen de la compra:**
 - Cantidad de GAIA adquiridos (con formato legible, ej. 1,250 GAIA).
 - Monto pagado en USDC (ej. 250 USDC).
 - Fecha y hora de la transacción en **hora local del usuario** y en **UTC**.
3. **Detalles de la transacción on-chain:**
 - TX ID (hash) en formato acortado (ej. 5x...k9p) con un botón para copiar al portapapeles.
 - Enlace a **Solscan** para ver la transacción en el explorador.
4. **Estado del vesting:**
 - Mensaje claro: "Tus tokens están bloqueados por 6 meses."
 - Fecha exacta del primer desbloqueo (25 %).
 - Calendario de liberación: fechas de los próximos desbloques (4 liberaciones mensuales).
 - Enlace al contrato de Streamflow donde se gestiona el vesting (si está disponible públicamente).
5. **Llamada a la acción (CTA):**
 - Botón: " Ver mi dashboard de inversión" que lleve al área privada del usuario.
 - Botón secundario: " Ver en Solscan" (ya incluido en el detalle).

3.1.2 Ejemplo Visual del Mensaje de Éxito

```

+-----+
| ¡Compra exitosa! |
| |
| Has adquirido 1,250 GAIA |
| |
| Pagado: 250 USDC |
| Fecha/Hora: 13 de agosto de 2026 - 09:32 (UTC: 14:32) |
| |
| Transacción: 5x...k9p [ Copiar] [ Ver en Solscan] |
| |
| ===== |
| |
| Vesting activo |
| • Bloqueo total: 6 meses (hasta el 13 de febrero de 2027) |
| • 1er desbloqueo (25%): 13 de febrero de 2027 |
| • 2do desbloqueo (25%): 13 de marzo de 2027 |
| • 3er desbloqueo (25%): 13 de abril de 2027 |
| • 4to desbloqueo (25%): 13 de mayo de 2027 |
| |
| Ver mi dashboard de inversión |
+-----+

```

3.2 Mensaje de Error

Si la transacción falla, el mensaje debe ser igual de claro y accionable.

3.2.1 Ejemplo Visual del Mensaje de Error

```

+-----+
| Transacción no completada |
| |
| Motivo: Slippage excedido |
| |
| Por favor, intenta de nuevo con un slippage mayor o reduce la |
+-----+

```

```

| cantidad de GAIA a comprar. |
|                               |
| [ Reintentar ]               |
| [ Ayuda ]                   |
+-----+

```

3.3 Estados de Error Comunes y Mensajes Asociados

Cuadro 2: Mensajes de Error por Motivo

Estado	Mensaje al Usuario
Slippage excedido	"El precio cambió durante la transacción. Ajusta el slippage o intenta con una cantidad menor."
Saldo insuficiente	"No tienes suficiente USDC en tu wallet para esta compra."
Error de red	"Hubo un problema de conexión con la red Solana. Por favor, verifica tu conexión y vuelve a intentarlo."
Rechazo del usuario	"Transacción cancelada por el usuario."
Error del contrato	"Error interno en el contrato de venta. Nuestro equipo ha sido notificado. Por favor, intenta más tarde."

3.4 Mensaje de Estado "Pendiente" (Timeout)

Si la transacción no se confirma en 60 segundos, la landing debe mostrar un estado de **pendiente**:

```

+-----+
| Tu transacción está siendo procesada por la red. |
|                                                  |
| TX ID: 5x...k9p                               |
|                                                  |
| Puedes verificar su estado en Solscan:         |
| [ Ver en Solscan]                             |
|                                                  |
| El estado se actualizará automáticamente en breve. |

```

Ver mi dashboard de inversión	
+-----+	

4 Enlace al Explorador y Datos On-Chain

4.1 Elección del Explorador

Se utilizará **Solscan** como explorador principal por su mejor experiencia de usuario para no técnicos. Solscan ofrece:

- Interfaz visual más amigable que Solana Explorer.
- Vista de tokens y balances asociados a la transacción.
- Información detallada del bloque y confirmaciones.

URL de respaldo: Solana Explorer ([https://explorer.solana.com/tx/\[TX_ID\]](https://explorer.solana.com/tx/[TX_ID])), para usuarios que prefieran la herramienta oficial.

4.2 Generación del Enlace

El enlace debe generarse dinámicamente con el TX ID capturado de la transacción:

`https://solscan.io/tx/\[TX\_ID\]`

Ejemplo: `https://solscan.io/tx/5x8f...k9p`

4.3 Visualización del Enlace en la UI

El enlace debe mostrarse de dos formas:

1. **Como botón principal:** " Ver en Solscan" – al hacer clic, abre una nueva pestaña con la transacción.
2. **Como texto del TX ID acortado:** ej. 5x...k9p, con un ícono de copiar al portapapeles.

4.4 Datos Adicionales On-Chain

Además del TX ID, la confirmación puede incluir:

- **Saldo de GAIA en la wallet del usuario después de la compra:** se puede consultar on-chain mediante el RPC de Solana.
- **Confirmaciones de la red:** número de bloques confirmados (mínimo 32 recomendado para considerar la transacción finalizada).

5 Gestión del Vesting (Streamflow)

5.1 Integración con Streamflow

El vesting de GAIA se gestionará mediante **Streamflow**, una plataforma de gestión de tokens bloqueados en Solana. Streamflow permite:

- Crear contratos de vesting con schedules personalizados.
- Visualizar el estado del vesting en tiempo real.
- Reclamar tokens cuando se desbloquean.

5.2 Flujo de Creación del Vesting

1. Después de la compra, el backend de Gaia debe comunicarse con la API de Streamflow para crear un contrato de vesting a nombre del usuario.
2. El contrato debe reflejar el schedule definido: **6 meses de lock + 4 meses de liberación escalonada (25 % mensual)**.
3. Streamflow devuelve un `vault_address` o identificador del contrato.
4. Este identificador se almacena en la base de datos junto con la transacción.
5. El usuario puede ver el estado de su vesting en el dashboard de Gaia, y también puede consultarlo directamente en la interfaz de Streamflow.

5.3 Enlace al Contrato de Streamflow

La confirmación de compra debe incluir un enlace a Streamflow para que el usuario pueda verificar el estado de su vesting de forma independiente:

`https://app.streamflow.finance/vesting/[VAULT_ADDRESS]`

Si Streamflow no ofrece una vista pública directa, al menos se debe mostrar el `vault_address` y una explicación de que el vesting está gestionado por Streamflow.

5.4 Mensaje Sugerido para el Vesting

Tu vesting está gestionado por Streamflow

- Contrato: [vault_address]
- Puedes ver el estado de tu bloqueo en cualquier momento en el dashboard de Gaia.

6 Dashboard del Usuario (“Mi Inversión”)

6.1 Propósito

El usuario debe poder regresar en cualquier momento y consultar el estado de su inversión sin tener que buscar el correo de confirmación o el historial de transacciones. El dashboard es un área privada accesible después de conectar la wallet.

6.2 Contenido del Dashboard

- **Resumen de la compra:**
 - Cantidad total de GAIA adquiridos.
 - Monto pagado en USDC.
 - Fecha de compra.
 - TX ID (con enlace a Solscan).
- **Estado del vesting:**
 - Gráfico de barras o timeline mostrando las fechas de liberación.
 - Cantidad liberada vs. bloqueada en cada momento.
 - Fechas de los próximos desbloques.
 - Botón para “Reclamar” los tokens liberados (integración con Streamflow).
- **Enlaces rápidos:**
 - Ver en Solscan.
 - Ver en Streamflow (estado del vesting).

6.3 Requisitos Técnicos

- El dashboard debe consultar el estado del vesting en tiempo real desde la API de Streamflow o desde el contrato on-chain.
- Debe mostrar el saldo actual de GAIA en la wallet del usuario (consultado mediante RPC de Solana).

- Si el usuario aún no ha comprado, debe mostrar un mensaje: "Aún no tienes inversiones en Gaia Ecotrack."

7 Consideraciones Técnicas para Meteora

7.1 Integración con Meteora

La preventa de GAIA se realizará a través de un pool de liquidez en Meteora. La integración debe manejar:

1. **Conexión de wallet:** Usar el Solana Wallet Adapter estándar (Phantom, Solflare, Backpack).
2. **Interacción con el pool:** El usuario selecciona la cantidad de GAIA a comprar y confirma la transacción. Meteora ejecuta el swap y devuelve el TX ID.
3. **Captura del TX ID:** La aplicación debe escuchar el evento de transacción completada y capturar el hash.
4. **Espera de confirmaciones:** Recomendado esperar al menos 1 confirmación (o hasta 32 bloques para mayor seguridad) antes de mostrar el mensaje de éxito.

7.2 Manejo de Slippage

El pool de Meteora es dinámico. Para evitar errores por slippage:

- Permitir al usuario ajustar el slippage (configuración por defecto: 0.5 % - 1 %).
- Mostrar una advertencia si el slippage es demasiado bajo.
- Si la transacción falla por slippage, mostrar el mensaje de error correspondiente y sugerir un ajuste.

7.3 Webhooks y Callbacks

Si Meteora ofrece webhooks o callbacks para notificar transacciones completadas, utilizarlos para actualizar automáticamente el estado de la transacción en el backend. Si no, se debe implementar un polling del RPC para detectar la confirmación.

7.4 Manejo de Fallos del RPC y Webhooks

La integración con Meteora y el RPC de Solana (Helius) puede experimentar latencia o fallos temporales. El manual debe especificar un comportamiento claro para estos escenarios.

7.4.1 Flujo de Confirmación con Polling

1. **Captura local del TX ID:** Inmediatamente después de que el usuario firma la transacción, la wallet devuelve un objeto de respuesta que contiene el `tx_id`. La aplicación debe capturar este ID **localmente**, sin depender exclusivamente del webhook de Meteora.
2. **Estado "Procesando":** La landing muestra un estado intermedio: " **Procesando** tu transacción... Esto puede tomar unos segundos."
3. **Polling al RPC:** La aplicación inicia un polling al RPC de Solana (o al backend, que a su vez consulta el RPC) para verificar el estado de la transacción usando el `tx_id`. Frecuencia recomendada: cada 2-3 segundos.
4. **Condición de éxito:** Si el RPC confirma la transacción (estado `confirmed` o `finalized`), la landing muestra el mensaje de éxito.
5. **Condición de timeout:** Si pasan 60 segundos sin confirmación, la landing muestra un estado de **pendiente** en lugar de un error definitivo (ver sección 3.4).
6. **Respaldo:** El backend debe continuar monitoreando la transacción en segundo plano y actualizar el dashboard del usuario cuando se confirme.

7.4.2 Implementación Técnica

- Utilizar la `getTransaction` del RPC de Solana con el parámetro `commitment: 'confirmed'`.
- Almacenar el `tx_id` y el estado en la base de datos con un campo `status` que puede ser: `pending`, `confirmed`, `failed`.
- Si el webhook de Meteora llega después de que el polling ya confirmó la transacción, ignorar el duplicado (idempotencia).

7.5 Seguridad

- Validar que la transacción se haya ejecutado en el contrato correcto (dirección del pool de Meteora).
- Verificar que el usuario haya recibido la cantidad esperada de GAIA.
- Implementar un límite máximo de compra por wallet para evitar abusos.

8 Plan de Pruebas y Lanzamiento

8.1 Fases de Pruebas

1. **Pruebas en Devnet:** Simular el flujo completo de compra, confirmación, y vesting en Solana Devnet con tokens de prueba.
2. **Pruebas en Testnet:** Repetir en Solana Testnet con un pool de Meteora de prueba.
3. **Pruebas de integración:** Verificar la comunicación entre la landing, el backend, Streamflow y Meteora.
4. **Pruebas de UX:** Realizar pruebas con usuarios reales (no técnicos) para validar la claridad de los mensajes y la facilidad de uso.
5. **Pruebas de carga:** Simular múltiples compras simultáneas para verificar la estabilidad del backend y de la integración con Meteora.

8.2 Checklist de Lanzamiento

La landing muestra correctamente el mensaje de confirmación después de cada compra.

El TX ID se captura y almacena en el backend.

El enlace a Solscan funciona correctamente.

El mensaje de vesting muestra las fechas correctas según el schedule (4 liberaciones).

El dashboard del usuario muestra los datos correctos de compra y vesting.

El contrato de Streamflow se crea correctamente para cada usuario.

Los mensajes de error se muestran en todos los escenarios de fallo.

El estado "pendiente" se muestra correctamente cuando el RPC tarda en confirmar.

La experiencia de Claim (Capítulo 9) funciona correctamente: botón dinámico, barra de progreso, historial.

Las pruebas de carga no muestran degradación del rendimiento.

La documentación de soporte está actualizada con los mensajes de error y pasos de solución.

8.3 Roles y Responsabilidades

Cuadro 3: Equipo Responsable

Rol		Responsabilidad
Desarrollador Frontend (Nico)		Implementación de la landing, el dashboard y la vista de Claim, integración con wallet adapter, Meteora y Streamflow.
Desarrollador Backend		API para almacenar transacciones, integración con Streamflow, y consulta de estado de vesting.
QA		Pruebas de flujo completo, validación de mensajes y estados de error.
Producto		Validación de la UX y copy de los mensajes de confirmación y claim.
Legal / Compliance		Verificación de que los mensajes de vesting y claim cumplan con las obligaciones legales de transparencia.

9 Experiencia de Reclamo (Claim) en el Dashboard

9.1 Objetivo del Capítulo

Definir los requerimientos de interfaz y lógica técnica para la página de reclamo de tokens (Claim). El objetivo principal es brindar total certidumbre visual al inversor sobre el estado de sus fondos gestionados en Streamflow, previniendo errores de ejecución y reduciendo la carga de soporte.

9.2 Indicadores Visuales de Estado y Tiempo

Para mitigar la ansiedad del usuario durante el período de vesting, el dashboard debe incluir elementos visuales precisos alimentados en tiempo real por el contrato de Streamflow.

9.2.1 Contador Regresivo Personalizado (Por Wallet)

El frontend debe leer el timestamp del próximo evento de desbloqueo del contrato de Streamflow asociado a la wallet conectada y mostrar un contador exacto en formato **Días : Horas : Minutos : Segundos** hasta ese momento.

Consideración técnica: Si el usuario ya ha reclamado todas sus liberaciones, el contador debe mostrar " **Todas las liberaciones completadas**".

Implementación: Utilizar la API de Streamflow para obtener el array de liberaciones (**releases**) y calcular la fecha del próximo evento que aún no ha ocurrido.

9.2.2 Barra de Progreso de Vesting

Una representación gráfica (barra horizontal o gráfico circular) que divida el total de la inversión en tres estados claramente diferenciados por color:

- **Bloqueado (Locked):** Tokens sujetos al cliff de 6 meses o aún no liberados. Color: gris.
- **Disponible (Claimable):** Porcentaje que ya cumplió el ciclo y está listo para retiro. Color: verde.

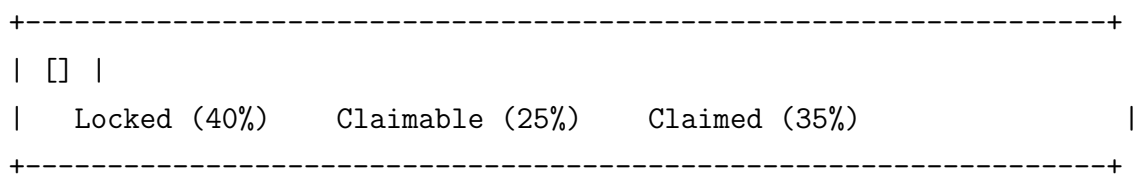
- **Reclamado (Claimed):** Cantidad histórica que el usuario ya ha transferido a su wallet. Color: azul.

Cálculo de los tres estados a partir de datos de Streamflow:

Cuadro 4: Cálculo de Estados desde Streamflow

Concepto	Fórmula
totalAmount	Total de tokens bloqueados en el contrato (monto inicial de la compra).
unlockedAmount	Cantidad de tokens que ya han superado el lock y están disponibles para reclamar (incluye los ya reclamados).
withdrawnAmount	Cantidad de tokens que el usuario ya ha reclamado (retirados a su wallet).
Claimable	$\text{unlockedAmount} - \text{withdrawnAmount}$ (lo que está disponible ahora).
Claimed	withdrawnAmount (lo que ya se retiró).
Locked	$\text{totalAmount} - \text{unlockedAmount}$ (lo que aún no se ha liberado).

Ejemplo visual de la barra de progreso:



9.3 Interfaz de Acción (Smart Buttons)

La interacción principal de retiro debe estar protegida contra ejecuciones fallidas para ahorrar gas y evitar frustraciones.

9.3.1 Botón de "Claim" Dinámico

- **Estado Inactivo:** Si el saldo Claimable es 0, el botón debe estar deshabilitado (grisado) y mostrar el texto: "Próximo desbloqueo en: [Contador]".

- **Estado Activo:** Automáticamente, al cumplirse el timestamp de liberación (es decir, cuando `Claimable > 0`), el botón cambia a estado activo (color primario) con el texto: "Reclamar [Cantidad] GAIA".
- **Validación de saldo de SOL:** Antes de habilitar el botón, el frontend debe consultar el saldo de SOL de la wallet. Si es inferior a 0.005 SOL (o un umbral definido), el botón se deshabilita y muestra un tooltip: "Saldo insuficiente de SOL para comisiones". Esta verificación previene el error más común en la transacción de claim.

9.3.2 Botón "Añadir Token a la Wallet" (One-Click)

Inmediatamente después de un claim exitoso, debe aparecer un botón secundario que permita importar el Mint Address de GAIA a Phantom, Solflare o Backpack. Esto previene el falso pánico de usuarios que reclaman con éxito pero no ven el saldo reflejado en su interfaz porque no han agregado el contrato del token.

Implementación: Usar el método `addToken` del wallet adapter (disponible en Phantom y Solflare). El Mint Address de GAIA debe ser configurado como constante en el frontend.

Comportamiento: Si el token ya está en la lista de la wallet, el botón debe estar oculto o mostrar un mensaje "Token ya agregado".

9.4 Transparencia y Prevención de Errores

9.4.1 Historial de Reclamos

Una tabla inferior en la vista de Claim que registre cada retiro realizado por esa wallet, con los siguientes campos:

- Fecha del reclamo (en hora local y UTC).
- Cantidad retirada (GAIA).
- Enlace directo al TX ID en Solscan.

Fuente de datos: Los reclamos se registran en el backend de Gaia al momento de ejecutar la transacción de claim. El historial se almacena en la base de datos y se muestra en el dashboard.


```
| 13/02/2027      | 312.5 GAIA | 4x...k9q  [ Ver en Solscan]      |
| 13/03/2027      | 312.5 GAIA | 7f...m3p  [ Ver en Solscan]      |
+-----+-----+-----+-----+-----+
```

9.6 Requisitos Técnicos para Desarrollo (Nico y equipo)

9.6.1 Lectura de Streamflow

- Utilizar la SDK o API de Streamflow para obtener el objeto del contrato del usuario a partir de su `vault_address`.
- Extraer los parámetros `totalAmount`, `unlockedAmount`, `withdrawnAmount` y la matriz de `releases`.
- Calcular los tres estados (Locked, Claimable, Claimed) según las fórmulas de la sección 9.2.2.
- Calcular el próximo evento de desbloqueo a partir de la matriz de `releases`.

9.6.2 Manejo de Errores en Claim

Si la transacción de claim falla, capturar el error del RPC y mostrar un mensaje claro y accionable:

Cuadro 5: Mensajes de Error en Claim

Causa del Error	Mensaje al Usuario
Falta de SOL para comisiones	"No tienes suficiente SOL para pagar la comisión de red. Deposita al menos 0.005 SOL en tu wallet."
Rechazo de la transacción por el usuario	"Transacción cancelada por el usuario."
Error de red / RPC sin respuesta	"La red Solana está congestionada. Por favor, intenta de nuevo en unos segundos."
Error interno del contrato de Streamflow	"Error en el contrato de vesting. Nuestro equipo ha sido notificado. Por favor, intenta más tarde."

9.6.3 Actualización en Tiempo Real

- Implementar un polling cada 30 segundos a la API de Streamflow para actualizar el estado del vesting.
- Si el usuario realiza un claim exitoso, el frontend debe refrescar inmediatamente los datos (sin esperar el próximo poll) para reflejar el nuevo saldo.
- El historial de reclamos debe almacenarse en el backend y consultarse mediante una API propia de Gaia.

9.7 Consideraciones de Seguridad

- Verificar que la wallet conectada coincida con la propietaria del contrato de vesting (evitar que un usuario intente reclamar tokens de otro).
- Validar que el monto reclamado no exceda el saldo disponible (evitar errores por datos desactualizados).
- Implementar límite de tasa (rate limiting) en el endpoint de claim para prevenir ataques de fuerza bruta.

10 Anexos

10.1 Anexo A: Ejemplo de Mensaje de Confirmación de Compra (Copy Exacto)

¡Compra exitosa!

Has adquirido [CANTIDAD] GAIA

Pagado: [MONTO] USDC

Transacción: [TX_ID]

Fecha/Hora: [FECHA_HORA_LOCAL] (UTC: [FECHA_HORA_UTC])

Vesting activo

- Bloqueo total: 6 meses (hasta el [FECHA_LOCK])
- 1er desbloqueo (25%): [FECHA1]
- 2do desbloqueo (25%): [FECHA2]
- 3er desbloqueo (25%): [FECHA3]
- 4to desbloqueo (25%): [FECHA4]

Ver mi dashboard de inversión

Nota técnica: El campo [FECHA_HORA_LOCAL] debe calcularse en el frontend usando la zona horaria del navegador del usuario. El campo (UTC: ...) debe mostrar la hora en formato UTC estándar (ISO 8601).

10.2 Anexo B: Estructura de Datos para el Backend

Listing 1: Estructura JSON de la Transacción de Compra

```
{
  "wallet": "GxY...p9Q",
  "tx_id": "5x8f...k9p",
  "amount_gaia": 1250,
  "amount_usdc": 250,
  "timestamp": "2026-08-13T14:32:00Z",
  "vesting_schedule": {
```

```

    "lock_until": "2027-02-13T00:00:00Z",
    "releases": [
      { "percentage": 25, "date": "2027-02-13T00:00:00Z" },
      { "percentage": 25, "date": "2027-03-13T00:00:00Z" },
      { "percentage": 25, "date": "2027-04-13T00:00:00Z" },
      { "percentage": 25, "date": "2027-05-13T00:00:00Z" }
    ]
  },
  "streamflow_vault": "vault_123456"
}

```

10.3 Anexo C: Estructura de Datos para el Historial de Reclamos

Listing 2: Estructura JSON de un Registro de Claim

```

{
  "wallet": "GxY...p9Q",
  "claim_tx_id": "4x7f...m9k",
  "amount_claimed": 312.5,
  "timestamp": "2027-02-13T10:15:00Z",
  "vesting_release_index": 0 // índice de la liberación (0-3)
}

```

10.4 Anexo D: Enlaces Útiles para Implementación

- Solscan API: <https://public-api.solscan.io/>
- Streamflow API Docs: <https://docs.streamflow.finance/>
- Meteora SDK: <https://docs.meteora.ag/>
- Solana Wallet Adapter: <https://github.com/solana-labs/wallet-adapter>
- Solana RPC (Helius): <https://docs.helius.dev/>

11 Referencias y Documentos Relacionados

1. Whitepaper Gaia Ecotrack v2.0 – Secciones 5.1.3 y 5.2.4 (Vesting y Tokenomics).
2. Manual de Estilo UX – Gaia Ecotrack (documento interno).
3. Guía de Integración con Meteora – Equipo de Meteora.
4. Guía de Integración con Streamflow – Equipo de Streamflow.